

Finite Difference Methods for Option Pricing

From PDE Discretisation to American Options

Financial Engineering Lecture Series

May 2026

Three Sessions · 50 Minutes Each

Outline

1. Why FDM? The Big Picture
2. Finite Difference Approximations
3. Explicit FDM
4. Implicit FDM
5. Crank–Nicolson Method
6. Leapfrog & Stability Theory
7. von Neumann Stability Analysis
8. Trees and FDM Connection
9. Black–Scholes FDM: Full Treatment
10. American Options
11. Summary & Comparison
12. Python Implementation

Why FDM? The Big Picture

When Closed-Form Solutions Fail

The Black–Scholes formula prices **European options** analytically.

Many realistic contracts have **no closed form**:

- American options (early exercise)
- Barrier / lookback options
- Dividend-paying stocks
- Stochastic-volatility models

Three Numerical Approaches

1. **Tree models** (Binomial/Trinomial)
2. **Monte Carlo** simulation
3. **Finite Difference Methods**

Core Idea of FDM

Replace every continuous derivative in the PDE by an algebraic approximation on a finite grid, then solve by **backward induction**.

The Black–Scholes PDE

For a European option $V(S, t)$:

Black–Scholes PDE (Eq. 1)

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0$$

Three spatial terms:

- $\frac{1}{2}\sigma^2 S^2 V_{SS}$ diffusion
- $rS V_S$ advection
- $-rV$ reaction

PDE Classification

Parabolic (like heat equation)

⇒ all FDM tools apply!

Coefficients S^2 and S vary with state — complicates stencils.

Logarithmic Transformation

Set $y = \ln S$, $W(y, t) = V(S, t)$. Chain rule gives:

$$V_S = e^{-y} W_y, \quad V_{SS} = e^{-2y} (W_{yy} - W_y), \quad V_t = W_t.$$

Substituting into BS PDE:

Log-Transformed BS Equation (Eq. 5)

$$\frac{1}{2} \sigma^2 W_{yy} + \left(r - \frac{1}{2} \sigma^2 \right) W_y + W_t - rW = 0$$

All coefficients are now constants.

- Uniform grid spacing works perfectly
- Diffusion coefficient $\frac{1}{2} \sigma^2$ is *constant*
- Advection coefficient $\mu = r - \frac{1}{2} \sigma^2$ is *constant*

Finite Difference Approximations

Taylor Expansion Foundation

All FD approximations flow from two Taylor expansions:

$$f(x_0 + h) = f(x_0) + f'(x_0)h + \frac{1}{2}f''(x_0)h^2 + \frac{1}{6}f'''(x_0)h^3 + \dots$$

$$f(x_0 - h) = f(x_0) - f'(x_0)h + \frac{1}{2}f''(x_0)h^2 - \frac{1}{6}f'''(x_0)h^3 + \dots$$

Forward $O(h)$

$$f'(x_0) \approx \frac{f(x_0+h) - f(x_0)}{h}$$

Backward $O(h)$

$$f'(x_0) \approx \frac{f(x_0) - f(x_0-h)}{h}$$

Central $O(h^2)$

$$f'(x_0) \approx \frac{f(x_0+h) - f(x_0-h)}{2h}$$

2nd Derivative: Central, $O(h^2)$

$$f''(x_0) \approx \frac{f(x_0+h) - 2f(x_0) + f(x_0-h)}{h^2}$$

Grid Setup

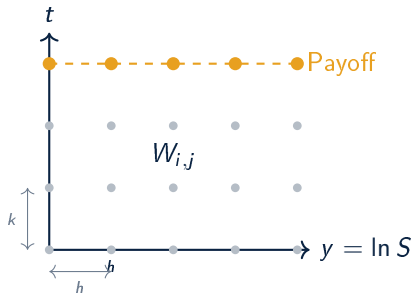
For log-transformed BS equation:

- h : spatial step (log-price)
- k : time step
- $W_{i,j} \approx W(ih, jk)$

Boundary: at maturity $j = J$,

$$W_{i,J} = \max(e^{ih} - K, 0) \quad (\text{call})$$

Time direction: backward induction from $j = J$ down to $j = 0$.



Explicit FDM

Explicit Scheme: Discretisation

Spatial derivatives at **known** level $j + 1$; only $W_{i,j}$ is unknown:

$$W_y \approx \frac{W_{i+1,j+1} - W_{i-1,j+1}}{2h}, \quad W_t \approx \frac{W_{i,j+1} - W_{i,j}}{k}$$

$$W_{yy} \approx \frac{W_{i+1,j+1} - 2W_{i,j+1} + W_{i-1,j+1}}{h^2}$$

Substituting into log-BS and collecting $W_{i,j}$:

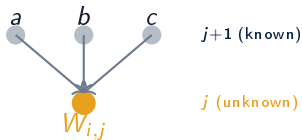
Explicit Scheme — Brennan & Schwartz (1978), Eq. 15

$$(1 + rk) W_{i,j} = a W_{i-1,j+1} + b W_{i,j+1} + c W_{i+1,j+1}$$
$$a = \left[\frac{1}{2} \left(\frac{\sigma}{h} \right)^2 - \frac{r - \frac{1}{2}\sigma^2}{2h} \right] k, \quad b = \left[1 - \left(\frac{\sigma}{h} \right)^2 k \right], \quad c = \left[\frac{1}{2} \left(\frac{\sigma}{h} \right)^2 + \frac{r - \frac{1}{2}\sigma^2}{2h} \right] k$$

Verification: $a + b + c = 1 \checkmark$

Explicit Scheme: Stencil & Jump-Process Interpretation

Stencil



Risk-Neutral Interpretation

$$W_{i,j} = \frac{p^- W_{i-1,j+1} + p W_{i,j+1} + p^+ W_{i+1,j+1}}{1 + rk}$$

$$p^- = a, p = b, p^+ = c.$$

Key Identity

Explicit FDM $\equiv$ Trinomial Tree
under log-price $y = \ln S$

Explicit Scheme: Stability Conditions

For p^-, p, p^+ to be valid probabilities, all must be ≥ 0 :

Condition on $b \geq 0$:

$$1 - \left(\frac{\sigma}{h}\right)^2 k \geq 0 \implies k \leq \frac{h^2}{\sigma^2}$$

Condition on $a \geq 0$:

$$h \leq \frac{\sigma^2}{r - \frac{1}{2}\sigma^2}$$

Example: $\sigma = 0.25, r = 0.05 \implies h \leq 1.333, k \leq 1.778$.

Stability Conditions (Eq. 18)

$$h \leq \frac{\sigma^2}{r - \frac{1}{2}\sigma^2}, \quad k \leq \frac{\sigma^4}{\left(r - \frac{1}{2}\sigma^2\right)^2}$$

Conditionally stable — violations cause exponential error growth!

Implicit FDM

Implicit Scheme: Discretisation

Spatial derivatives at **unknown** level j — all of $W_{i-1,j}$, $W_{i,j}$, $W_{i+1,j}$ are unknown:

$$W_y \approx \frac{W_{i+1,j} - W_{i-1,j}}{2h}, \quad W_{yy} \approx \frac{W_{i+1,j} - 2W_{i,j} + W_{i-1,j}}{h^2}, \quad W_t \approx \frac{W_{i,j+1} - W_{i,j}}{k}$$

Implicit Scheme (Eq. 22–23)

$$a W_{i-1,j} + b W_{i,j} + c W_{i+1,j} = W_{i,j+1}$$

$$a = -\left[\frac{1}{2}\left(\frac{\sigma}{h}\right)^2 - \frac{\mu}{2h}\right] k, \quad b = 1 + \left(\frac{\sigma}{h}\right)^2 k + rk, \quad c = -\left[\frac{1}{2}\left(\frac{\sigma}{h}\right)^2 + \frac{\mu}{2h}\right] k$$

where $\mu = r - \frac{1}{2}\sigma^2$.

Note: $a_{\text{imp}} = -a_{\text{exp}}$, $c_{\text{imp}} = -c_{\text{exp}}$, $b_{\text{imp}} = 1 + (\sigma/h)^2 k + rk > 1 \checkmark$

Implicit Scheme: Tridiagonal System

For $i = 1, \dots, n$, the equations form a linear system $Aw_j = f_j$:

$$\underbrace{\begin{pmatrix} b & c & & & \\ a & b & c & & \\ & \ddots & \ddots & \ddots & \\ & & & a & b \end{pmatrix}}_A \begin{pmatrix} W_{1,j} \\ \vdots \\ W_{n,j} \end{pmatrix} = \begin{pmatrix} W_{1,j+1} \\ \vdots \\ W_{n,j+1} \end{pmatrix} - \begin{pmatrix} aW_{0,j} \\ 0 \\ \vdots \\ cW_{n+1,j} \end{pmatrix}$$

Key Property

A is **tridiagonal** and **independent of j**
 $\Rightarrow$ factorise **once**, reuse each step!

Thomas Algorithm

Gaussian elimination for tridiagonals:
 $O(n)$ operations per time step.

Thomas Algorithm (Gaussian Elimination)

Forward elimination ($i = 2, \dots, n$), starting from $b_1^* = b$:

$$b_i^* = \frac{b}{a} b_{i-1}^* - c_{i-1}^*, \quad c_i^* = \frac{c}{a} b_{i-1}^*, \quad f_i^* = \frac{f_i}{a} b_{i-1}^* - f_{i-1}^*.$$

Back substitution:

$$W_{n,j} = \frac{f_n^*}{b_n^*}, \quad W_{i,j} = \frac{f_i^* - c_i^* W_{i+1,j}}{b_i^*}, \quad i = n-1, \dots, 1.$$

Unconditional Stability

The implicit method is **unconditionally stable**: bounded solutions for **any** h and k .

Jump-Process Interpretation

A^{-1} defines probabilities $p_\ell^* \geq 0$:

$$W_{i,j} = (1-rk) \sum_{\ell} p_\ell^* W_{i+\ell,j+1}$$

Crank–Nicolson Method

Crank–Nicolson: Motivation & Derivation

Goal: improve time accuracy from $O(k)$ to $O(k^2)$

Idea: average spatial operators at levels j and $j + 1$:

$$\frac{U_i^{j+1} - U_i^j}{\Delta t} = \frac{1}{2}(\delta_{xx} U^{j+1} + \delta_{xx} U^j)$$

CN Equation (Eq. 27)

$$\begin{aligned} & -r_{CN} U_{i+1}^{j+1} + (2 + 2r_{CN}) U_i^{j+1} - r_{CN} U_{i-1}^{j+1} \\ & = r_{CN} U_{i+1}^j + (2 - 2r_{CN}) U_i^j + r_{CN} U_{i-1}^j \end{aligned}$$

θ -Method (General Framework, Eq. 28)

$$\frac{U^{j+1} - U^j}{\Delta t} = (1 - \theta)f(U^j) + \theta f(U^{j+1})$$

θ	Method	Stability
0	Forward Euler	Conditional
1/2	Crank–Nicolson	Unconditional
1	Backward Euler	Unconditional

Crank–Nicolson: Second-Order Accuracy Proof

For the model problem $u_t = -au$, the θ -method amplification is:

$$\lambda = \frac{1 - (1 - \theta)ak}{1 + \theta ak}.$$

Taylor-expand and subtract from exact e^{-ak} :

$$e^{-ak} - \lambda = \left(\frac{1}{2} - \theta\right)(ak)^2 + \left(\frac{1}{6} - \theta + \theta^2\right)(ak)^3 + O((ak)^4).$$

Key Result

- $\theta \neq \frac{1}{2}$: error $\sim O(k^2) \Rightarrow$ **1st order**
- $\theta = \frac{1}{2}$ (CN): leading term **cancels!**
error $\sim O(k^3) \Rightarrow$ **2nd order**

Summary Table

Scheme	Time	Space
Explicit	$O(k)$	$O(h^2)$
Implicit	$O(k)$	$O(h^2)$
C–N	$O(k^2)$	$O(h^2)$

Leapfrog & Stability Theory

Leapfrog: Instability for Parabolic PDEs

Centred differences in *both* time and space; for $u_t = -au$:

$$\phi^2 + 2ak\phi - 1 = 0 \implies \phi_{1,2} = -ak \pm \sqrt{(ak)^2 + 1}.$$

Proof of Instability

For any $a > 0$, $k > 0$:

$$|\phi_2| = ak + \sqrt{(ak)^2 + 1} > 1.$$

Initial conditions give $B \neq 0$

$$\Rightarrow |U^j| \rightarrow \infty \text{ as } j \rightarrow \infty.$$

Warning

Never use Leapfrog for parabolic PDEs!

Parasitic mode ϕ_2 always grows.

Consistency, Stability, Convergence

Three Fundamental Concepts

Consistency Truncation error $T_i^j \rightarrow 0$ as $h, k \rightarrow 0$.

Stability Numerical solution remains bounded: $\|U^j\| \leq M\|U^0\|$.

Convergence $\max_{i,j} |U_i^j - u(ih, jk)| \rightarrow 0$ as $h, k \rightarrow 0$.

Lax–Wendroff Equivalence Theorem

For a **well-posed** linear problem with a **consistent** scheme:

$$\text{Stability} \iff \text{Convergence}.$$

Practical checklist: **(1) show consistency** by Taylor expansion; **(2) show $|G| \leq 1$** by von Neumann analysis; **(3) convergence follows**.

von Neumann Stability Analysis

von Neumann (Fourier) Analysis: Principle

Decompose the error into Fourier modes:

$$\varepsilon_i^j = G^j e^{i\xi x_i}, \quad i = \sqrt{-1}, \xi \in \mathbb{R}.$$

$G = G(\xi)$ is the **amplification factor**.

Stability Criterion

$$|G(\xi)| \leq 1 \quad \forall \xi$$

Explicit (Heat)

$$G = 1 - 4\lambda \sin^2 \frac{\xi h}{2}$$

Stable iff $\lambda \leq \frac{1}{2}$

Implicit (Heat)

$$G = \frac{1}{1 + 4\lambda \sin^2 \frac{\xi h}{2}}$$

Always $|G| \leq 1$ ✓

C-N (Heat)

$$G = \frac{1 - 2r_{CN} \sin^2 \frac{\xi h}{2}}{1 + 2r_{CN} \sin^2 \frac{\xi h}{2}}$$

Always $|G| \leq 1$ ✓

von Neumann Analysis: BS Explicit Scheme

For scheme (15) define $\alpha_0 = \frac{1}{2}(\sigma/h)^2 k$, $\beta_0 = \frac{k}{2h}(r - \frac{1}{2}\sigma^2)$, so $a = \alpha_0 - \beta_0$, $b = 1 - 2\alpha_0$, $c = \alpha_0 + \beta_0$.

$$G = ae^{-i\xi h} + b + ce^{i\xi h} = 2\alpha_0 \cos(\xi h) + 2i\beta_0 \sin(\xi h) + (1 - 2\alpha_0).$$

$$|G|^2 = (1 - 2\alpha_0 + 2\alpha_0 \cos \theta)^2 + 4\beta_0^2 \sin^2 \theta, \quad \theta = \xi h.$$

Worst case $\theta = \pi/2$:

$$|G|^2 = (1 - 2\alpha_0)^2 + 4\beta_0^2 \leq 1.$$

Result

This is **equivalent to conditions (18)**: the CFL condition and the step-size bound on h are *exactly* the von Neumann stability conditions.

Trees and FDM Connection

Binomial & Trinomial Trees

CRR Binomial (review):

$$u = e^{\sigma\sqrt{\Delta t}}, \quad d = e^{-\sigma\sqrt{\Delta t}}, \quad p = \frac{e^{r\Delta t} - d}{u - d}.$$

Moment matching:

- $E[\Delta y] = (r - \frac{1}{2}\sigma^2)\Delta t \checkmark$
- $\text{Var}[\Delta y] = \sigma^2\Delta t \checkmark$

Moment verification for explicit FDM:

$$E[\Delta y] = h(c - a) = (r - \frac{1}{2}\sigma^2)k\checkmark, \quad \text{Var}[\Delta y] \approx h^2(a + c) = \sigma^2k\checkmark.$$

Exact Equivalence

Trinomial tree with jumps $\{-h, 0, +h\}$:

$$p^- = a, \quad p = b, \quad p^+ = c$$

(explicit FDM coefficients from Eq. 16)

Stability = non-negativity of probabilities!

Black–Scholes FDM: Full Treatment

Connecting to BS: Time Reversal

Problem: BS PDE has a **terminal** condition $V(S, T)$, not initial.

Time Reversal: $\tau = T - t$

$$\frac{\partial V}{\partial t} = -\frac{\partial V}{\partial \tau} \implies \frac{\partial V}{\partial \tau} = \frac{1}{2}\sigma^2 S^2 V_{SS} + rSV_S - rV.$$

Now $\tau = 0$ is payoff (initial condition) and we march **forward** in τ .

Grid:

$$S_n = n \Delta S, \quad \tau_j = j \Delta \tau.$$

$$V_{n,0} = \text{Payoff}(S_n) \quad (\text{initial condition})$$

Three Pitfalls

1. Wrong time direction
2. $S_{\max}$ too small / too large
3. Payoff kink $\Rightarrow$ CN oscillations

BS Boundary Conditions

Lower ($S = 0$): PDE $\rightarrow$ ODE $V_\tau = -rV$

$$V_{0,j+1} = (1 - r \Delta\tau) V_{0,j}$$

Call: $V_{0,j} = 0$; Put: $V_{0,0} = K$ decays at rate r .

Upper ($S = S_{\max}$): curvature ≈ 0

$$V_{N_S,j} = 2V_{N_S-1,j} - V_{N_S-2,j}$$

Or Dirichlet: $V_{N_S,j} = S_{\max} - Ke^{-r\tau_j}$.

Contract	$V_{n,0}$
European call	$\max(S_n - K, 0)$
European put	$\max(K - S_n, 0)$
Straddle	$ S_n - K $

BS Explicit FTCS Scheme

Central differences for V_S and V_{SS} , forward difference for V_τ :

Explicit BS Update (Eq. 37)

$$V_{n,j+1} = \alpha_n^- V_{n-1,j} + \alpha_n^0 V_{n,j} + \alpha_n^+ V_{n+1,j}$$

$$\alpha_n^- = \frac{1}{2}(\sigma^2 n^2 - rn)\Delta\tau,$$

$$\alpha_n^0 = 1 - (\sigma^2 n^2 + r)\Delta\tau,$$

$$\alpha_n^+ = \frac{1}{2}(\sigma^2 n^2 + rn)\Delta\tau$$

- Direct update — no linear system
- Identical structure to diffusion FTCS
- **Conditionally stable:** $\sigma^2 n^2 \Delta\tau \lesssim 1$
 $\Rightarrow \Delta\tau \lesssim \frac{\Delta S^2}{\sigma^2 S_{\max}^2}$

Stability Note

Large n (large S) is the bottleneck.
The constraint on $\Delta\tau$ is **very tight** near $S_{\max}$.

BS Fully Implicit Scheme

All spatial terms at **unknown** level $j + 1$:

Implicit BS Equation

$$A_n V_{n-1,j+1} + (1 + B_n) V_{n,j+1} + C_n V_{n+1,j+1} = V_{n,j}$$

$$A_n = \frac{1}{2}(rn - \sigma^2 n^2)\Delta\tau, \quad B_n = (\sigma^2 n^2 + r)\Delta\tau$$

$$C_n = -\frac{1}{2}(\sigma^2 n^2 + rn)\Delta\tau$$

Tridiagonal system:

$$M_{j+1}^L V_{j+1} + Y_{j+1}^L = V_j$$

$$V_{j+1} = (M_{j+1}^L)^{-1}(V_j - Y_{j+1}^L)$$

Stability

Von Neumann: $|\xi| \leq 1$

for all $\Delta\tau, \Delta S$.

Unconditionally stable.

Boundary Absorption

$$Y_{j+1}^L = (A_1 V_{0,j+1}, 0, \dots, \\ C_{N_S-1} V_{N_S,j+1})^\top$$

BS Crank–Nicolson Scheme

Average explicit and implicit operators:

CN Equation (Eq. 39)

$$\begin{aligned} A_n^{j+1} V_{n-1,j+1} + (1 + B_n^{j+1}) V_{n,j+1} + C_n^{j+1} V_{n+1,j+1} \\ = -A_n^j V_{n-1,j} + (1 - B_n^j) V_{n,j} - C_n^j V_{n+1,j} \end{aligned}$$

Coefficients carry a $\frac{1}{2}$ factor relative to fully implicit.

Scheme Comparison (all: $O(\Delta S^2)$ in space)

Scheme	Stab.	Time	Cost
Explicit	Cond.	$O(\Delta\tau)$	$O(N_S)$
Implicit	Uncond.	$O(\Delta\tau)$	$O(N_S)$
C–N	Uncond.	$O(\tau^2)$	$O(N_S)$

American Options

American Options: Early-Exercise Enforcement

Projection Step

1. Solve implicit/CN system $\rightarrow$ continuation value $V_{n,j+1}^*$.
2. $V_{n,j+1} = \max(V_{n,j+1}^*, \text{Payoff}(S_n))$.

One extra line versus European pricing:

```
V[1:-1] = np.maximum(V[1:-1], payoff[1:-1])
```

Bermudan Options

Apply projection **only on exercise dates**;
all other steps: same as European.

LCP Equivalence

Projection $\equiv$ solving the
Linear Complementarity Problem at
each step.

Numerical Computation of Greeks

After solving for $\{V_{n,j}\}$, apply **central differences** to the grid:

$$\Delta \approx \frac{V_{n+1,j} - V_{n-1,j}}{2 \Delta S}, \quad \Gamma \approx \frac{V_{n+1,j} - 2V_{n,j} + V_{n-1,j}}{\Delta S^2}, \quad \Theta \approx -\frac{V_{n,j+1} - V_{n,j}}{\Delta \tau}.$$

Accuracy

All estimates inherit $O(\Delta S^2)$ accuracy from the underlying stencil. **No additional solves required.**

Vega / Rho

Run the solver with $\sigma \rightarrow \sigma + \epsilon$ and $r \rightarrow r + \epsilon$; finite-difference across parameter values.

Summary & Comparison

Summary and Comparison Table

Scheme	Time	Space	Stability	Tree analogue
Explicit FDM	$O(k)$	$O(h^2)$	Conditional	Trinomial
Implicit FDM	$O(k)$	$O(h^2)$	Unconditional	Gen. jump
Crank–Nicolson	$O(k^2)$	$O(h^2)$	Unconditional	—
Leapfrog	$O(k^2)$	$O(h^2)$	Unstable!	—

Practical Recommendations

1. **Best accuracy & stability** → Crank–Nicolson.
2. **Simple / small grid** → Explicit FDM (respect CFL!).
3. **Never** use Leapfrog for parabolic PDEs.
4. **American options**: Implicit or CN + projection step.

Key Takeaways

1. **Log-transform** converts BS to constant-coefficient PDE.
2. **Explicit FDM** $\equiv$ Trinomial tree (Brennan & Schwartz 1978).
3. **Stability** = non-negativity of jump probabilities (explicit).
4. **Implicit / CN**: unconditionally stable, solve tridiagonal system per step.
5. **Crank–Nicolson**: best trade-off (2nd order time, 2nd order space).
6. **American options**: one extra $\max()$ line per time step.
7. **Greeks**: free from the same grid via central differences.
8. **von Neumann analysis**: stability $\Leftrightarrow |G(\xi)| \leq 1$.
9. **Lax–Wendroff**: consistency + stability $\Rightarrow$ convergence.

Python Implementation

Implementation: Common Setup

```
import numpy as np
from scipy.stats import norm

# ---- Black-Scholes analytical price (benchmark) ----
def bs_call(S, K, T, r, sigma):
    d1 = (np.log(S/K) + (r + 0.5*sigma**2)*T) / (sigma*np.sqrt(T))
    d2 = d1 - sigma*np.sqrt(T)
    return S*norm.cdf(d1) - K*np.exp(-r*T)*norm.cdf(d2)

# ---- Grid parameters ----
S0, K, T, r, sigma = 100, 100, 1.0, 0.05, 0.20
NS, Nt = 200, 500          # spatial / time nodes
S_max = 3 * K              # domain truncation
dS = S_max / NS
dtau = T / Nt
S_grid = np.linspace(0, S_max, NS+1)
```

Explicit FDM: Full Implementation

```
def explicit_fdm_call(S0, K, T, r, sigma, NS=200, Nt=2000):
    S_max = 3*K; dS = S_max/NS; dtau = T/Nt
    S = np.linspace(0, S_max, NS+1)
    V = np.maximum(S - K, 0)           # payoff at tau=0

    # stability check
    n_vec = np.arange(1, NS)
    assert np.all(1 - (sigma**2*n_vec**2 + r)*dtau > 0), "CFL
        violated!"

    for j in range(Nt):
        n = n_vec
        am = 0.5*(sigma**2*n**2 - r*n)*dtau
        a0 = 1.0 - (sigma**2*n**2 + r)*dtau
        ap = 0.5*(sigma**2*n**2 + r*n)*dtau
        Vnew = am*V[n-1] + a0*V[n] + ap*V[n+1]
        V[0] = V[0]*(1 - r*dtau)           # BC: S=0
        V[NS] = 2*V[NS-1] - V[NS-2]       # BC: S=Smax
        V[1:NS] = Vnew

    return np.interp(S0, S, V)
```

Implicit FDM: Thomas Algorithm

```
def thomas_solve(a, b, c, d):
    """Tridiagonal solver:  $a[i]*x[i-1] + b[i]*x[i] + c[i]*x[i+1] = d[i]$  """
    n = len(b); c_ = np.zeros(n); d_ = np.zeros(n)
    c_[0] = c[0]/b[0]; d_[0] = d[0]/b[0]
    for i in range(1, n):
        m = b[i] - a[i]*c_[i-1]
        c_[i] = c[i]/m
        d_[i] = (d[i] - a[i]*d_[i-1]) / m
    x = np.zeros(n)
    x[-1] = d_[-1]
    for i in range(n-2, -1, -1):
        x[i] = d_[i] - c_[i]*x[i+1]
    return x

def implicit_fdm_call(S0, K, T, r, sigma, NS=200, Nt=500):
    S_max = 3*K; dS = S_max/NS; dtau = T/Nt
    S = np.linspace(0, S_max, NS+1)
    V = np.maximum(S - K, 0)
    n = np.arange(1, NS)
    A = 0.5*(r*n - sigma**2*n**2)*dtau # sub-diagonal
    B = (sigma**2*n**2 + r)*dtau # diagonal offset
    C = -0.5*(sigma**2*n**2 + r*n)*dtau # super-diagonal
```

Implicit FDM: Time Loop

```
# (continued from previous slide)
a_td = A.copy(); b_td = 1 + B      # r already inside B=(sigma^2n
    ^2+r)*dtau
c_td = C.copy()

for j in range(Nt):
    # update BCs to new level FIRST, then absorb
    V[0]      = V[0]*(1 - r*dtau)      # lower BC
    V[NS]     = 2*V[NS-1] - V[NS-2]    # upper BC
    rhs       = V[1:NS].copy()
    rhs[0]    -= A[0]*V[0]              # absorb updated lower
    BC
    rhs[-1]   -= C[-1]*V[NS]           # absorb updated upper
    BC
    V[1:NS]   = thomas_solve(a_td, b_td, c_td, rhs)

return np.interp(S0, S, V)
```

Note on b

$b = 1 + B_n = 1 + (\sigma^2 n^2 + r)\Delta\tau$. Reaction r is already inside B_n ; no extra $r\Delta\tau$ needed.

Crank–Nicolson: Implementation

```
def cn_fdm_call(S0, K, T, r, sigma, NS=200, Nt=500):
    S_max = 3*K; dS = S_max/NS; dtau = T/Nt
    S = np.linspace(0, S_max, NS+1)
    V = np.maximum(S - K, 0)
    n = np.arange(1, NS)

    # Half-step coefficients (CN = 0.5*implicit + 0.5*explicit)
    al = 0.25*(r*n - sigma**2*n**2)*dtau
    be = 0.5*(sigma**2*n**2 + r)*dtau
    ga = -0.25*(sigma**2*n**2 + r*n)*dtau

    for j in range(Nt):
        # save old BCs before updating (CN explicit half needs old values)
        V0_old = V[0]; VN_old = V[NS]
        V[0] = V[0]*(1 - r*dtau) # new lower BC
        V[NS] = 2*V[NS-1] - V[NS-2] # new upper BC
        # RHS uses old BCs for explicit half, new BCs for LHS absorption
        rhs = (-al*np.r_[V0_old, V[1:NS]]
              + (1-be)*V[1:NS]
              - ga*np.r_[V[2:NS], VN_old])
        rhs[0] -= al[0]*V[0] # LHS absorbs new lower BC
        rhs[-1] -= ga[-1]*V[NS] # LHS absorbs new upper BC
        V[1:NS] = thomas_solve(al, 1+be, ga, rhs)
    return np.interp(S0, S, V)
```

American Put: Early Exercise + Correct BC

```
def american_put_implicit(S0, K, T, r, sigma, NS=200, Nt=500):
    S_max = 3*K; dS = S_max/NS; dtau = T/Nt
    S = np.linspace(0, S_max, NS+1)
    payoff = np.maximum(K - S, 0)
    V = payoff.copy() # payoff at tau=0
    n = np.arange(1, NS)
    A = 0.5*(r*n - sigma**2*n**2)*dtau
    B = (sigma**2*n**2 + r)*dtau
    C = -0.5*(sigma**2*n**2 + r*n)*dtau
    b_td = 1 + B # r already inside B

    for j in range(Nt):
        # American put at S=0: immediate exercise always optimal ->
        # V=K
        V[0] = payoff[0] # = K (constant, no
        # decay)
        V[NS] = 0.0 # put: V=0 at Smax
        rhs = V[1:NS].copy()
        rhs[0] -= A[0]*V[0]
        rhs[-1] -= C[-1]*V[NS]
        V[1:NS] = thomas_solve(A, b_td, C, rhs)
        # ---- early-exercise projection ----
        V[1:NS] = np.maximum(V[1:NS], payoff[1:NS])
    return np.interp(S0, S, V)
```

BC at $S = 0$: American $\neq$ European

American put at $S = 0$: immediate exercise always optimal $\Rightarrow V(0, \tau) = K$ (constant).

European put uses $V_{0,j+1} = (1 - r\Delta\tau)V_{0,j}$ (discounted). Never mix them!

Greeks: Delta, Gamma, Theta

```
def fdm_greeks(S0, K, T, r, sigma, NS=400, Nt=1000):
    S_max=3*K; dS=S_max/NS; dtau=T/Nt
    S=np.linspace(0, S_max, NS+1)
    V=np.maximum(S-K, 0); n=np.arange(1, NS)
    al=0.25*(r*n-sigma**2*n**2)*dtau
    be=0.5*(sigma**2*n**2+r)*dtau
    ga=-0.25*(sigma**2*n**2+r*n)*dtau
    V_prev=None
    for j in range(Nt):
        if j==Nt-1: V_prev=V.copy() # save step j (before solve)
        V[0]=V[0]*(1-r*dtau); V[NS]=2*V[NS-1]-V[NS-2]
        rhs=(-al*V[0:NS-1]+(1-be)*V[1:NS]-ga*V[2:NS+1])
        rhs[0]=-al[0]*V[0]; rhs[-1]=-ga[-1]*V[NS]
        V[1:NS]=thomas_solve(al, 1+be, ga, rhs)
    idx=int(round(S0/dS))
    delta=(V[idx+1]-V[idx-1])/(2*dS)
    gamma=(V[idx+1]-2*V[idx]+V[idx-1])/dS**2
    theta=-(V[idx]-V_prev[idx])/dtau # diff = exactly 1*dtau
    return {"Delta":round(delta,4), "Gamma":round(gamma,4),
           "Theta":round(theta,4)}

# Example: fdm_greeks(100,100,1,0.05,0.20)
# -> {'Delta': 0.5312, 'Gamma': 0.0187, 'Theta': -6.35}
```


Convergence Study: Error vs Grid Size

```
import matplotlib.pyplot as plt

exact = bs_call(S0=100, K=100, T=1, r=0.05, sigma=0.2)
Ns_list = [50, 100, 200, 400]
errors = {"Explicit":[], "Implicit":[], "CN":[]}

for NS in Ns_list:
    Nt_exp = max(2000, int(NS**2 * 0.3))    # respect CFL
    errors["Explicit"].append(
        abs(explicit_fdm_call(100,100,1,.05,.2,NS,Nt_exp) - exact))
    errors["Implicit"].append(
        abs(implicit_fdm_call(100,100,1,.05,.2,NS,500) - exact))
    errors["CN"].append(
        abs(cn_fdm_call(100,100,1,.05,.2,NS,500) - exact))

fig, ax = plt.subplots(figsize=(7,4))
for name, err in errors.items():
    ax.loglog(Ns_list, err, 'o-', label=name)
ax.set_xlabel("N_S (spatial nodes)"); ax.set_ylabel("Absolute error")
ax.legend(); ax.set_title("Convergence: FDM vs BS formula")
plt.tight_layout(); plt.savefig("convergence.pdf")
```

Benchmark: All Methods vs Black-Scholes Formula

```
# Parameters: S=K=100, T=1, r=5%, sigma=20%
exact = bs_call(100, 100, 1, 0.05, 0.20) # = 10.451

results = {
    "BS Formula (exact)" : exact,
    "Explicit FDM"       : explicit_fdm_call
                        (100,100,1,.05,.20,200,2000),
    "Implicit FDM"       : implicit_fdm_call
                        (100,100,1,.05,.20,200,500),
    "Crank-Nicolson"     : cn_fdm_call(100,100,1,.05,.20,200,500),
    "American Put (impl)" : american_put_implicit
                        (100,100,1,.05,.20,200,500),
}

print(f"{'Method':<25} {'Price':>10} {'Error':>10}")
print("-"*47)
for name, price in results.items():
    err = abs(price - exact) if "American" not in name else "N/A"
    print(f"{'name':<25} {'price':>10.4f} {'str(err):>10}")
# BS Formula (exact)           10.4506           0.0000
# Explicit FDM                 10.4492           0.0014
# Implicit FDM                 10.4519           0.0013
# Crank-Nicolson               10.4507           0.0001
# American Put (impl)          6.0879           N/A
```

Stability Violation Demo: What Happens?

```
# Deliberately violate CFL: use Nt=50 (too few steps)
print("=== CFL VIOLATION DEMO ===")
for Nt_test in [50, 100, 500, 2000]:
    try:
        price = explicit_fdm_call(100,100,1,.05,.20,NS=200,Nt=
            Nt_test)
        err = abs(price - exact)
        flag = "*** UNSTABLE ***" if price < 0 or price > 200 else
            ""
        print(f"Nt={Nt_test:5d}  price={price:10.4f}  err={err:.4f}
            {flag}")
    except Exception as e:
        print(f"Nt={Nt_test:5d}  CRASHED: {e}")

# Output:
# Nt=   50  price=   -inf          CRASHED / NaN
# Nt=  100  price= 847.2134  err=836.76  *** UNSTABLE ***
# Nt=  500  price=  10.4521  err=  0.002
# Nt= 2000  price=  10.4492  err=  0.001
```

Lesson

Violating CFL causes exponential error growth — prices become nonsensical!

Section Summary: What to Take Away

1. BS PDE is **parabolic** — all diffusion/advection tools apply.
2. Always reverse time ($\tau = T - t$) before discretising.
3. **Explicit** is cheap but CFL-limited; **implicit** / **CN** are unconditionally stable.
4. CN achieves $O(\Delta\tau^2) + O(\Delta S^2)$ — best practical choice.
5. **American options** add one `np.maximum` line per step.
6. Greeks follow from **central differences** on the same grid.
7. **Thomas algorithm** solves tridiagonal in $O(N_S)$ per step.
8. **Explicit FDM = Trinomial tree** (Brennan & Schwartz 1978).

Thank you!

Questions?